topic_4_object_pool

Theory, Edge Cases & Code Examples

- ✓ Object Pool Pattern Overview
- ✓ Object Pool vs Memory Pool
- ✓ Object Pools in Autonomous Vehicles
- ✓ Double-Free Detection
- ✓ Thread-Safety Without Proper Synchronization

→ *Swipe to learn* →

topic_4_object_pool

Theory

📌 1. Object Pool Pattern Overview (1/2)

Definition: Pre-allocates and manages a collection of reusable objects, lending them to clients instead of creating/destroying on demand.

Core Concept:

```
// Simplified Object Pool
template <typename T, size_t PoolSize>
class ObjectPool {
    T storage[PoolSize];           // Pre-allocated objects
    std::vector<T*> freeList;      // Available objects
    bool inUse[PoolSize] = {};     // Track allocation state

public:
    ObjectPool() {
        // Initialize free list with all objects
        for (size_t i = 0; i < PoolSize; ++i) {
            freeList.push_back(&storage[i]);
        }
    }

    T* allocate() {
        if (freeList.empty()) return nullptr;
        T* obj = freeList.back();
        freeList.pop_back();
        return obj;  // ✅ O(1) allocation
    }

    void deallocate(T* obj) {
        freeList.push_back(obj);  // ✅ O(1) deallocation
    }
};
```

topic_4_object_pool

Theory

📌 1. Object Pool Pattern Overview (2/2)

Lifecycle Comparison:

Operation	Traditional (new/delete)	Object Pool
Initial cost	None	Pre-allocate all objects
Per-allocation	Heap allocation (~100ns)	Pop from free list (~5ns)
Per-deallocation	Heap deallocation (~100ns)	Push to free list (~5ns)
Fragmentation	❌ Increases over time	✅ Zero (fixed memory)
Timing	⚠️ Unpredictable (GC, OS)	✅ Deterministic O(1)
Cache locality	⚠️ Scattered	✅ Contiguous array

Speedup: 10-100x faster than heap allocation for small, frequently-created objects.

topic_4_object_pool

Theory

📌 2. Object Pool vs Memory Pool (1/2)

Abstraction Levels:

Aspect	Object Pool	Memory Pool
Manages	Constructed objects (T)	Raw memory blocks (void*)
Type safety	✅ Strongly typed	❌ Type-agnostic
Construction	Objects pre-constructed	Requires placement new
Destruction	Optional reset/reuse	Manual destructor calls
Use case	Single type, frequent reuse	Multiple types, custom allocators
Interface	`allocate()` returns T*	`allocate(size)` returns void*

Object Pool Example:

```
// Object Pool - type-specific, objects ready
ObjectPool<SensorReading, 1000> sensorPool;

auto* reading = sensorPool.allocate();  // ✅ Ready to use
reading->timestamp = now();
reading->value = 42.0;

sensorPool.deallocate(reading);  // ✅ Object returned to pool
```

topic_4_object_pool

Theory

📌 2. Object Pool vs Memory Pool (2/2)

Memory Pool Example:

```
// Memory Pool - raw bytes, placement new required
MemoryPool pool(1000 * sizeof(SensorReading));

void* mem = pool.allocate(sizeof(SensorReading));
SensorReading* reading = new (mem) SensorReading();  // Placement new

reading->~SensorReading();  // Manual destructor
pool.deallocate(mem);
```

Decision Matrix:

```
Same type allocated repeatedly? ──YES──> Object Pool
         │
         NO
         ↓
Multiple types, custom allocator? ──YES──> Memory Pool
         │
         NO
         ↓
Just use heap allocation (new/delete)
```

topic_4_object_pool

Theory

📌 3. Object Pools in Autonomous Vehicles (1/5)

Critical Use Cases:

Component	Frequency	Pool Size	Performance Requirement
LiDAR point clouds	10Hz × 100k points	~1M points	< 10ms processing
Radar detections	20Hz × 100 objects	~2k objects	< 5ms latency
Particle filters	100Hz × 1k particles	~100k particles	< 1ms per cycle
Path planning nodes	On-demand, ~10k/sec	~50k nodes	< 100μs per node
Message buffers	1kHz inter-process	~10k buffers	< 10μs allocation

topic_4_object_pool

Theory

📌 3. Object Pools in Autonomous Vehicles (2/5)

Performance Benefits:

A. Real-Time Determinism:

```
// ❌ Traditional heap allocation - unpredictable timing
void processLidarFrame() {
    for (int i = 0; i < 100000; ++i) {
        auto* point = new Point3D();  // ⚠️ May trigger GC, fragmentation
        processPoint(point);
        delete point;  // ⚠️ Timing varies: 50-500ns
    }
}
// Total: 5-50ms (10x variance!)

// ✅ Object pool - deterministic timing
ObjectPool<Point3D, 100000> pointPool;

void processLidarFrame() {
    for (int i = 0; i < 100000; ++i) {
        auto* point = pointPool.allocate();  // ✅ Always ~5ns
        processPoint(point);
        pointPool.deallocate(point);  // ✅ Always ~5ns
    }
}
// Total: ~1ms (consistent)
```

B. Zero Fragmentation:

Scenario	Traditional Heap	Object Pool
After 1 hour operation	Fragmented, 20% overhead	Contiguous, 0% overhead
Allocation success rate	May fail due to fragmentation	Always succeeds (if available)
Memory layout	Scattered across heap	Sequential array
Cache misses	⚠️ High (scattered)	✅ Low (contiguous)

topic_4_object_pool

Theory

📌 3. Object Pools in Autonomous Vehicles (3/5)

C. Cache-Friendly Memory Access:

```
// Object pool uses contiguous array
Point3D storage[100000];  // Sequential memory

// Accessing nearby objects benefits from cache prefetching
for (int i = 0; i < 100000; ++i) {
    process(&storage[i]);  // ✅ Cache hit rate: ~95%
}

// vs heap allocation (scattered memory)
std::vector<Point3D*> heap_points;
for (auto* p : heap_points) {
    process(p);  // ⚠️ Cache hit rate: ~60% (scattered addresses)
}
```

topic_4_object_pool

Theory

📌 3. Object Pools in Autonomous Vehicles (4/5)

Real-World Application:

```
// Autonomous vehicle perception pipeline
class PerceptionSystem {
    // Pools for different object types
    ObjectPool<Point3D, 1'000'000> lidarPool;      // 1M point pool
    ObjectPool<RadarDetection, 10'000> radarPool;  // 10k detection pool
    ObjectPool<TrackedObject, 1'000> trackPool;    // 1k tracked object pool

public:
    void processFrame() {
        // LiDAR processing (10Hz, 100k points/frame)
        for (int i = 0; i < 100'000; ++i) {
            auto* point = lidarPool.allocate();  // ~5ns
            point->x = rawData[i].x;
            point->y = rawData[i].y;
            point->z = rawData[i].z;

            if (isObstacle(point)) {
                auto* detection = radarPool.allocate();
                detection->position = *point;
                detections.push_back(detection);
            }

            lidarPool.deallocate(point);  // ~5ns
        }

        // Object tracking
        for (auto* det : detections) {
            auto* obj = trackPool.allocate();
            updateTrack(obj, det);
            radarPool.deallocate(det);
        }

        // Timing: ~1ms total (deterministic)
        // vs ~10-50ms with heap allocation (variable)
    }
};
```

topic_4_object_pool

Theory

📌 3. Object Pools in Autonomous Vehicles (5/5)

Performance Metrics:

Metric	Heap Allocation	Object Pool	Improvement
Allocation time	~100ns	~5ns	20x faster
Deallocation time	~100ns	~5ns	20x faster
Timing variance	±10x (GC spikes)	±1%	Deterministic
Memory fragmentation	Increases over time	Zero	Stable
Cache miss rate	~40%	~5%	8x better locality

Critical Benefits:

- ✅ Meets hard real-time deadlines (control loops at 100Hz+)
- ✅ Prevents memory exhaustion in long-running systems
- ✅ Predictable performance for safety certification
- ✅ No dynamic allocation in critical paths

---

topic_4_object_pool

Edge Cases

📌 1. Double-Free Detection (1/2)

Problem: Deallocating the same pointer twice corrupts the free list, leading to double allocations and undefined behavior.

```
// ❌ Without protection
class UnsafePool {
    std::vector<T*> freeList;
public:
    void deallocate(T* ptr) {
        freeList.push_back(ptr);  // ❌ No duplicate check!
    }
};

ObjectPool pool;
T* obj = pool.allocate();
pool.deallocate(obj);
pool.deallocate(obj);  // ❌ Double-free! Pool corruption

T* a = pool.allocate();
T* b = pool.allocate();
// a == b! ❌ Both clients get same memory

// ✅ With protection using usage tracking
template <typename T, size_t N>
class SafePool {
    T* storage;
    std::vector<T*> freeList;
    bool used[N] = {};  // Track allocated slots

public:
    T* allocate() {
        if (freeList.empty()) throw std::bad_alloc();

        T* obj = freeList.back();
        freeList.pop_back();

        size_t index = obj - storage;
        used[index] = true;  // ✅ Mark as in-use
        return obj;
    }

    void deallocate(T* ptr) {
// ...continues on next slide
```

📌 1. Double-Free Detection (2/2)

[Code continues across slides]

```
// ...continued from previous slide
        size_t index = ptr - storage;

        if (index >= N) {
            throw std::invalid_argument("Pointer not from pool");
        }

        if (!used[index]) {
            throw std::logic_error("Double-free detected!");  // ✅ Catch error
        }

        used[index] = false;
        freeList.push_back(ptr);
    }
};
```

Key Takeaway: Always track object usage state with a boolean array or bitset to detect double-frees during development.

topic_4_object_pool

Edge Cases

📌 2. Thread-Safety Without Proper Synchronization (1/2)

Problem: Multiple threads accessing the pool concurrently without synchronization causes data races on the free list.

```
// ❌ Not thread-safe
class UnsafePool {
    std::vector<T*> freeList;
public:
    T* allocate() {
        if (freeList.empty()) throw std::bad_alloc();
        T* obj = freeList.back();  // ❌ Race condition
        freeList.pop_back();       // ❌ Concurrent modification
        return obj;
    }
};

// Thread 1 and Thread 2 call allocate() simultaneously
// Both read freeList.back() before either pops
// Both get the same pointer! ❌ Memory corruption

// ✅ Thread-safe with mutex
class ThreadSafePool {
    std::vector<T*> freeList;
    std::mutex poolMutex;

public:
    T* allocate() {
        std::lock_guard<std::mutex> lock(poolMutex);  // ✅ Serialize access

        if (freeList.empty()) throw std::bad_alloc();

        T* obj = freeList.back();
        freeList.pop_back();
        return obj;
    }

    void deallocate(T* ptr) {
        std::lock_guard<std::mutex> lock(poolMutex);  // ✅ Serialize access
        freeList.push_back(ptr);
    }
};

// ...continues on next slide
```

📌 2. Thread-Safety Without Proper Synchronization (2/2)

[Code continues across slides]

```
// ...continued from previous slide
// ✅ Lock-free for SPSC (Single Producer Single Consumer)
template <typename T, size_t N>
class LockFreePoolSPSC {
    T* storage;
    std::atomic<size_t> head{0};  // Producer writes here
    std::atomic<size_t> tail{0};  // Consumer reads here
    T* buffer[N];

public:
    T* allocate() {  // Consumer only
        size_t t = tail.load(std::memory_order_acquire);
        size_t h = head.load(std::memory_order_acquire);

        if (t == h) return nullptr;  // Empty

        T* obj = buffer[t % N];
        tail.store(t + 1, std::memory_order_release);
        return obj;
    }

    void deallocate(T* ptr) {  // Producer only
        size_t h = head.load(std::memory_order_relaxed);
        buffer[h % N] = ptr;
        head.store(h + 1, std::memory_order_release);
    }
};
```

Key Takeaway: Use `std::mutex` for general thread-safety or lock-free atomics for SPSC scenarios. Never access shared pool state without synchronization.

topic_4_object_pool

Edge Cases

📌 3. Chunk Expansion Index Tracking (1/2)

Problem: When pools expand dynamically with multiple chunks, index calculations must account for chunk boundaries.

```
// ❌ Wrong: Single chunk assumption
class BrokenExpandablePool {
    std::vector<T*> chunks;
    std::vector<size_t> freeList;  // Global indices

    T* allocate() {
        size_t index = freeList.back();
        freeList.pop_back();
        return &chunks[0][index];  // ❌ Assumes single chunk!
    }
};

// ✅ Correct: Chunk + offset calculation
class ExpandablePool {
    std::vector<T*> chunks;
    std::vector<size_t> freeList;  // Global indices
    static constexpr size_t CHUNK_SIZE = 100;

    T* allocate() {
        if (freeList.empty()) {
            allocateChunk();
        }

        size_t globalIndex = freeList.back();
        freeList.pop_back();

        // ✅ Calculate chunk and offset
        size_t chunkIndex = globalIndex / CHUNK_SIZE;
        size_t offset = globalIndex % CHUNK_SIZE;

        return &chunks[chunkIndex][offset];
    }

    void deallocate(T* ptr) {
        // ✅ Find which chunk contains this pointer
        for (size_t chunkIdx = 0; chunkIdx < chunks.size(); ++chunkIdx) {
            T* chunkStart = chunks[chunkIdx];
            T* chunkEnd = chunkStart + CHUNK_SIZE;
// ...continues on next slide
```

📌 3. Chunk Expansion Index Tracking (2/2)

[Code continues across slides]

```
// ...continued from previous slide

            if (ptr >= chunkStart && ptr < chunkEnd) {
                size_t offset = ptr - chunkStart;
                size_t globalIndex = chunkIdx * CHUNK_SIZE + offset;
                freeList.push_back(globalIndex);
                return;
            }
        }

        throw std::invalid_argument("Pointer not from pool");
    }

private:
    void allocateChunk() {
        T* chunk = new T[CHUNK_SIZE]();
        chunks.push_back(chunk);

        size_t baseIndex = (chunks.size() - 1) * CHUNK_SIZE;
        for (size_t i = 0; i < CHUNK_SIZE; ++i) {
            freeList.push_back(baseIndex + i);
        }
    }
};
```

Key Takeaway: In expandable pools, maintain global index space across chunks using `chunkIndex * CHUNK_SIZE + offset` calculations.

topic_4_object_pool

Edge Cases

📌 4. Object Destruction and Placement New (1/2)

Problem: For non-trivial types, failing to call destructors before returning objects to the pool causes resource leaks.

```
struct Resource {
    std::string name;
    std::vector<int> data;
    std::unique_ptr<int> ptr;

    Resource() : ptr(std::make_unique<int>(42)) {}
};

// ❌ Without proper destruction
class LeakyPool {
    Resource* storage;
    std::vector<Resource*> freeList;

public:
    void deallocate(Resource* obj) {
        freeList.push_back(obj);  // ❌ Destructor never called!
        // Memory leaks: string, vector, unique_ptr not released
    }

    Resource* allocate() {
        Resource* obj = freeList.back();
        freeList.pop_back();
        return obj;  // ❌ Returns object with stale state
    }
};

// ✅ Proper lifecycle management
class ProperPool {
    alignas(Resource) char storage[sizeof(Resource) * 100];
    Resource* freeList[100];
    size_t freeCount = 100;

public:
    ProperPool() {
        // Initialize free list with addresses
        for (size_t i = 0; i < 100; ++i) {
            freeList[i] = reinterpret_cast<Resource*>(storage + i * sizeof(Resource));
        }
// ...continues on next slide
```

📌 4. Object Destruction and Placement New (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    }

    Resource* allocate() {
        if (freeCount == 0) throw std::bad_alloc();

        Resource* obj = freeList[--freeCount];
        new (obj) Resource();  // ✅ Placement new - construct object
        return obj;
    }

    void deallocate(Resource* obj) {
        obj->~Resource();  // ✅ Explicitly call destructor
        freeList[freeCount++] = obj;
    }

    ~ProperPool() {
        // No objects should be allocated when pool destructs
        // If freeCount < 100, some objects are still in use (leak)
    }
};
```

Key Takeaway: For non-POD types, use placement new on allocation and explicit destructor calls on deallocation to manage object lifetimes properly.

topic_4_object_pool

Edge Cases

📌 5. Alignment Requirements (1/2)

Problem: Types with strict alignment (SIMD, atomics) require properly aligned memory that may not be guaranteed by `new char[]`.

```
// ❌ Potentially misaligned
struct alignas(32) SIMDData {
    float values[8];  // Requires 32-byte alignment for AVX
};

class MisalignedPool {
    char* storage = new char[sizeof(SIMDData) * 100];  // ❌ Only 1-byte aligned!

    SIMDData* allocate() {
        return reinterpret_cast<SIMDData*>(storage);  // ❌ Undefined behavior if misaligned
    }
};

// ✅ Properly aligned storage
class AlignedPool {
    alignas(32) char storage[sizeof(SIMDData) * 100];  // ✅ Aligned array

    // Or use aligned allocation
    void* allocateAligned(size_t size, size_t alignment) {
        void* ptr = nullptr;
        #ifdef _WIN32
            ptr = _aligned_malloc(size, alignment);
        #else
            ptr = aligned_alloc(alignment, size);
        #endif
        return ptr;
    }

public:
    SIMDData* storage;

    AlignedPool() {
        storage = static_cast<SIMDData*>(allocateAligned(sizeof(SIMDData) * 100, 32));
    }

    ~AlignedPool() {
// ...continues on next slide
```

📌 5. Alignment Requirements (2/2)

[Code continues across slides]

```
// ...continued from previous slide
        #ifdef _WIN32
            _aligned_free(storage);
        #else
            free(storage);
        #endif
    }
};

// ✅ C++17 aligned_storage
template <typename T, size_t N>
class ModernAlignedPool {
    using AlignedStorage = std::aligned_storage_t<sizeof(T), alignof(T)>;
    AlignedStorage storage[N];  // ✅ Correctly aligned

public:
    T* allocate(size_t index) {
        return reinterpret_cast<T*>(&storage[index]);  // ✅ Safe
    }
};
```

Key Takeaway: Always respect alignment requirements using `alignas`, `std::aligned_storage`, or platform-specific aligned allocation functions.

topic_4_object_pool

Code Examples

📌 1. Fixed-Size Object Pool (Easy) (1/3)

Key Concepts:
- Pre-allocated contiguous storage
- Free list using `std::vector<T*>`
- RAII for automatic cleanup
- Object reset on deallocation

```
#include <iostream>
#include <vector>
#include <stdexcept>

struct Particle {
    float x, y, z;
    float vx, vy, vz;
    float lifetime;
};

template <typename T, size_t PoolSize = 100>
class ObjectPool {
    T* storage;
    std::vector<T*> freeList;

public:
    ObjectPool() : storage(new T[PoolSize]()) {
        // Initialize free list with all objects
        for (size_t i = 0; i < PoolSize; ++i) {
            freeList.push_back(&storage[i]);
        }
    }

    ~ObjectPool() {
        delete[] storage;
    }

    // Disable copy and move
    ObjectPool(const ObjectPool&) = delete;
    ObjectPool& operator=(const ObjectPool&) = delete;

    T* allocate() {
// ...continues on next slide
```

topic_4_object_pool

Code Examples

📌 1. Fixed-Size Object Pool (Easy) (2/3)

```
// ...continued from previous slide
        if (freeList.empty()) {
            throw std::bad_alloc();
        }

        T* obj = freeList.back();
        freeList.pop_back();
        return obj;
    }

    void deallocate(T* obj) {
        // Optional: reset object state
        *obj = T();
        freeList.push_back(obj);
    }

    size_t available() const {
        return freeList.size();
    }

    size_t capacity() const {
        return PoolSize;
    }
};

int main() {
    ObjectPool<Particle, 1000> particlePool;

    std::cout << "Initial capacity: " << particlePool.capacity() << "\n";
    std::cout << "Available: " << particlePool.available() << "\n";

    // Allocate some particles
    Particle* p1 = particlePool.allocate();
    p1->x = 1.0f; p1->y = 2.0f; p1->z = 3.0f;
    p1->lifetime = 5.0f;

    Particle* p2 = particlePool.allocate();
    p2->x = 10.0f; p2->y = 20.0f; p2->z = 30.0f;

    std::cout << "After allocation: " << particlePool.available() << "\n";

// ...continues on next slide
```

topic_4_object_pool

Code Examples

📌 1. Fixed-Size Object Pool (Easy) (3/3)

[Code continues across slides]

```
// ...continued from previous slide
    // Return to pool
    particlePool.deallocate(p1);
    std::cout << "After deallocation: " << particlePool.available() << "\n";

    // Reuse
    Particle* p3 = particlePool.allocate();  // Likely reuses p1's slot
    std::cout << "p3 position: " << p3->x << ", " << p3->y << ", " << p3->z << "\n";

    return 0;
}

// Output:
// Initial capacity: 1000
// Available: 1000
// After allocation: 998
// After deallocation: 999
// p3 position: 0, 0, 0  (reset by deallocate)
```

topic_4_object_pool

Code Examples

📌 2. Index-Based Memory Pool (Mid) (1/4)

Key Concepts:
- Index-based tracking (more cache-friendly than pointers)
- Double-free detection with `used[]` array
- Pointer validation using pointer arithmetic
- Memory zeroing on allocation

```
#include <iostream>
#include <stdexcept>
#include <cstring>

struct SensorData {
    uint64_t timestamp;
    float values[16];
    uint8_t sensorId;
};

template <typename T, size_t PoolSize = 100>
class IndexBasedPool {
    T* storage;
    size_t freeList[PoolSize];  // Store indices, not pointers
    size_t freeCount;
    bool used[PoolSize];  // Track allocated slots

public:
    IndexBasedPool() : storage(new T[PoolSize]()), freeCount(PoolSize) {
        // Initialize free list with indices (LIFO order)
        for (size_t i = 0; i < PoolSize; ++i) {
            freeList[i] = PoolSize - 1 - i;
            used[i] = false;
        }
    }

    ~IndexBasedPool() {
        delete[] storage;
    }

    T* allocate() {
        if (freeCount == 0) {
// ...continues on next slide
```

topic_4_object_pool

Code Examples

📌 2. Index-Based Memory Pool (Mid) (2/4)

```
// ...continued from previous slide
            throw std::bad_alloc();
        }

        // Pop index from free list
        size_t index = freeList[--freeCount];
        used[index] = true;

        // Zero the memory
        std::memset(&storage[index], 0, sizeof(T));

        return &storage[index];
    }

    void deallocate(T* ptr) {
        // Calculate index from pointer
        size_t index = ptr - storage;

        // Validate pointer
        if (index >= PoolSize) {
            throw std::invalid_argument("Pointer not from this pool");
        }

        // Detect double-free
        if (!used[index]) {
            throw std::logic_error("Double-free detected!");
        }

        used[index] = false;

        // Push index back to free list
        freeList[freeCount++] = index;
    }

    size_t available() const { return freeCount; }

    // Debugging helper
    void printStats() const {
        std::cout << "Pool stats:\n";
        std::cout << "  Capacity: " << PoolSize << "\n";
        std::cout << "  Available: " << freeCount << "\n";
// ...continues on next slide
```

topic_4_object_pool

Code Examples

📌 2. Index-Based Memory Pool (Mid) (3/4)

```
// ...continued from previous slide
        std::cout << "  In use: " << (PoolSize - freeCount) << "\n";
    }
};

int main() {
    IndexBasedPool<SensorData, 5> pool;

    pool.printStats();

    // Allocate
    SensorData* s1 = pool.allocate();
    SensorData* s2 = pool.allocate();
    s1->sensorId = 1;
    s2->sensorId = 2;

    pool.printStats();

    // Deallocate
    pool.deallocate(s1);
    pool.printStats();

    // Try double-free (will throw)
    try {
        pool.deallocate(s1);
    } catch (const std::logic_error& e) {
        std::cout << "Caught: " << e.what() << "\n";
    }

    return 0;
}

// Output:
// Pool stats:
//   Capacity: 5
//   Available: 5
//   In use: 0
// Pool stats:
//   Capacity: 5
//   Available: 3
//   In use: 2
// ...continues on next slide
```

📌 2. Index-Based Memory Pool (Mid) (4/4)

[Code continues across slides]

```
// ...continued from previous slide
// Pool stats:
//   Capacity: 5
//   Available: 4
//   In use: 1
// Caught: Double-free detected!
```

topic_4_object_pool

Code Examples

📌 3. Thread-Safe Object Pool (Advanced) (1/3)

Key Concepts:
- `std::mutex` for mutual exclusion
- `std::lock_guard` for RAII-style locking
- Thread-safe allocation and deallocation
- Multiple producer threads sharing pool

```
#include <iostream>
#include <vector>
#include <mutex>
#include <thread>
#include <chrono>

struct Message {
    char data[256];
    size_t length;
    uint64_t timestamp;
};

template <typename T, size_t PoolSize>
class ThreadSafePool {
    T* storage;
    std::vector<T*> freeList;
    std::mutex poolMutex;

public:
    ThreadSafePool() : storage(new T[PoolSize]()) {
        for (size_t i = 0; i < PoolSize; ++i) {
            freeList.push_back(&storage[i]);
        }
    }

    ~ThreadSafePool() {
        delete[] storage;
    }

    T* allocate() {
        std::lock_guard<std::mutex> lock(poolMutex);

// ...continues on next slide
```

topic_4_object_pool

Code Examples

📌 3. Thread-Safe Object Pool (Advanced) (2/3)

```
// ...continued from previous slide
        if (freeList.empty()) {
            return nullptr;  // Non-throwing version
        }

        T* obj = freeList.back();
        freeList.pop_back();
        return obj;
    }

    void deallocate(T* ptr) {
        std::lock_guard<std::mutex> lock(poolMutex);
        freeList.push_back(ptr);
    }

    size_t available() const {
        std::lock_guard<std::mutex> lock(poolMutex);
        return freeList.size();
    }
};

// Worker thread function
void producer(ThreadSafePool<Message, 100>& pool, int threadId) {
    for (int i = 0; i < 50; ++i) {
        Message* msg = pool.allocate();

        if (msg) {
            msg->timestamp = std::chrono::steady_clock::now().time_since_epoch().count();
            snprintf(msg->data, sizeof(msg->data), "Thread %d - Message %d", threadId, i);
            msg->length = strlen(msg->data);

            // Simulate processing
            std::this_thread::sleep_for(std::chrono::milliseconds(1));

            pool.deallocate(msg);
        } else {
            std::cout << "Thread " << threadId << " - Pool exhausted\n";
        }
    }
}

// ...continues on next slide
```

topic_4_object_pool

Code Examples

📌 3. Thread-Safe Object Pool (Advanced) (3/3)

[Code continues across slides]

```
// ...continued from previous slide
int main() {
    ThreadSafePool<Message, 100> messagePool;

    std::cout << "Initial available: " << messagePool.available() << "\n";

    // Launch multiple threads
    std::vector<std::thread> threads;
    for (int i = 0; i < 4; ++i) {
        threads.emplace_back(producer, std::ref(messagePool), i);
    }

    // Wait for completion
    for (auto& t : threads) {
        t.join();
    }

    std::cout << "Final available: " << messagePool.available() << "\n";

    return 0;
}

// Output (typical):
// Initial available: 100
// Final available: 100
```

topic_4_object_pool

Code Examples

📌 4. Expandable Chunk-Based Pool (Advanced) (1/3)

Key Concepts:
- Dynamic growth with chunks
- Global index space across chunks
- Chunk + offset calculation
- Automatic expansion on exhaustion

```
#include <iostream>
#include <vector>
#include <mutex>

template <typename T>
class ExpandablePool {
    static constexpr size_t CHUNK_SIZE = 100;

    std::vector<T*> chunks;
    std::vector<size_t> freeList;  // Global indices
    std::mutex poolMutex;

public:
    ExpandablePool() {
        allocateChunk();
    }

    ~ExpandablePool() {
        for (T* chunk : chunks) {
            delete[] chunk;
        }
    }

    T* allocate() {
        std::lock_guard<std::mutex> lock(poolMutex);

        if (freeList.empty()) {
            allocateChunk();
        }

        size_t globalIndex = freeList.back();
        freeList.pop_back();
// ...continues on next slide
```

topic_4_object_pool

Code Examples

📌 4. Expandable Chunk-Based Pool (Advanced) (2/3)

```
// ...continued from previous slide

        size_t chunkIndex = globalIndex / CHUNK_SIZE;
        size_t offset = globalIndex % CHUNK_SIZE;

        return &chunks[chunkIndex][offset];
    }

    void deallocate(T* ptr) {
        std::lock_guard<std::mutex> lock(poolMutex);

        // Find which chunk contains this pointer
        for (size_t chunkIdx = 0; chunkIdx < chunks.size(); ++chunkIdx) {
            T* chunkStart = chunks[chunkIdx];
            T* chunkEnd = chunkStart + CHUNK_SIZE;

            if (ptr >= chunkStart && ptr < chunkEnd) {
                size_t offset = ptr - chunkStart;
                size_t globalIndex = chunkIdx * CHUNK_SIZE + offset;
                freeList.push_back(globalIndex);
                return;
            }
        }

        throw std::invalid_argument("Pointer not from pool");
    }

    size_t capacity() const {
        std::lock_guard<std::mutex> lock(poolMutex);
        return chunks.size() * CHUNK_SIZE;
    }

    size_t chunkCount() const {
        std::lock_guard<std::mutex> lock(poolMutex);
        return chunks.size();
    }

private:
    void allocateChunk() {
        T* chunk = new T[CHUNK_SIZE]();
        chunks.push_back(chunk);
// ...continues on next slide
```

topic_4_object_pool

Code Examples

📌 4. Expandable Chunk-Based Pool (Advanced) (3/3)

[Code continues across slides]

```
// ...continued from previous slide

        size_t baseIndex = (chunks.size() - 1) * CHUNK_SIZE;
        for (size_t i = 0; i < CHUNK_SIZE; ++i) {
            freeList.push_back(baseIndex + i);
        }

        std::cout << "Allocated chunk #" << chunks.size()
                  << " (capacity now: " << capacity() << ")\n";
    }
};

int main() {
    ExpandablePool<int> pool;

    std::vector<int*> allocated;

    // Allocate beyond initial capacity
    for (int i = 0; i < 250; ++i) {
        int* ptr = pool.allocate();
        *ptr = i;
        allocated.push_back(ptr);
    }

    std::cout << "Total chunks: " << pool.chunkCount() << "\n";
    std::cout << "Total capacity: " << pool.capacity() << "\n";

    // Deallocate all
    for (int* ptr : allocated) {
        pool.deallocate(ptr);
    }

    return 0;
}

// Output:
// Allocated chunk #1 (capacity now: 100)
// Allocated chunk #2 (capacity now: 200)
// Allocated chunk #3 (capacity now: 300)
// Total chunks: 3
// Total capacity: 300
```

topic_4_object_pool

Code Examples

📌 5. Aligned Memory Pool for SIMD (Advanced) (1/3)

Key Concepts:
- Platform-specific aligned allocation
- SIMD type support (AVX __m256)
- Alignment verification
- Proper cleanup with aligned_free

```
#include <iostream>
#include <cstdlib>
#include <cstring>
#include <immintrin.h>  // For AVX

// SIMD-friendly structure requiring 32-byte alignment
struct alignas(32) Vec8f {
    float data[8];

    void add(const Vec8f& other) {
        __m256 a = _mm256_load_ps(data);
        __m256 b = _mm256_load_ps(other.data);
        __m256 result = _mm256_add_ps(a, b);
        _mm256_store_ps(data, result);
    }
};

template <typename T, size_t PoolSize>
class AlignedPool {
    static_assert(alignof(T) <= 64, "Alignment too large");

    T* storage;
    T* freeList[PoolSize];
    size_t freeCount;

public:
    AlignedPool() : freeCount(PoolSize) {
        // Allocate aligned memory
        #ifdef _WIN32
            storage = static_cast<T*>(_aligned_malloc(sizeof(T) * PoolSize, alignof(T)));
        #else
            storage = static_cast<T*>(aligned_alloc(alignof(T), sizeof(T) * PoolSize));
// ...continues on next slide
```

📌 5. Aligned Memory Pool for SIMD (Advanced) (2/3)

```
// ...continued from previous slide
        #endif

        if (!storage) {
            throw std::bad_alloc();
        }

        // Initialize memory
        std::memset(storage, 0, sizeof(T) * PoolSize);

        // Populate free list
        for (size_t i = 0; i < PoolSize; ++i) {
            freeList[i] = &storage[i];
        }
    }

    ~AlignedPool() {
        #ifdef _WIN32
            _aligned_free(storage);
        #else
            free(storage);
        #endif
    }

    T* allocate() {
        if (freeCount == 0) {
            throw std::bad_alloc();
        }

        return freeList[--freeCount];
    }

    void deallocate(T* ptr) {
        freeList[freeCount++] = ptr;
    }

    // Verify alignment
    bool isAligned(void* ptr) const {
        return (reinterpret_cast<uintptr_t>(ptr) % alignof(T)) == 0;
    }
};
// ...continues on next slide
```

topic_4_object_pool

Code Examples

📌 5. Aligned Memory Pool for SIMD (Advanced) (3/3)

[Code continues across slides]

```
// ...continued from previous slide

int main() {
    AlignedPool<Vec8f, 10> pool;

    Vec8f* v1 = pool.allocate();
    Vec8f* v2 = pool.allocate();

    std::cout << "v1 aligned: " << pool.isAligned(v1) << "\n";
    std::cout << "v2 aligned: " << pool.isAligned(v2) << "\n";

    // Initialize
    for (int i = 0; i < 8; ++i) {
        v1->data[i] = i;
        v2->data[i] = i * 10;
    }

    // SIMD addition
    v1->add(*v2);

    std::cout << "Result: ";
    for (int i = 0; i < 8; ++i) {
        std::cout << v1->data[i] << " ";
    }
    std::cout << "\n";

    pool.deallocate(v1);
    pool.deallocate(v2);

    return 0;
}

// Output:
// v1 aligned: 1
// v2 aligned: 1
// Result: 0 11 22 33 44 55 66 77
```

What You Learned

Key Takeaways

- ✓ Object Pool Pattern Overview
- ✓ Object Pool vs Memory Pool
- ✓ Object Pools in Autonomous Vehicles
- ✓ Double-Free Detection

→ Swipe to learn →